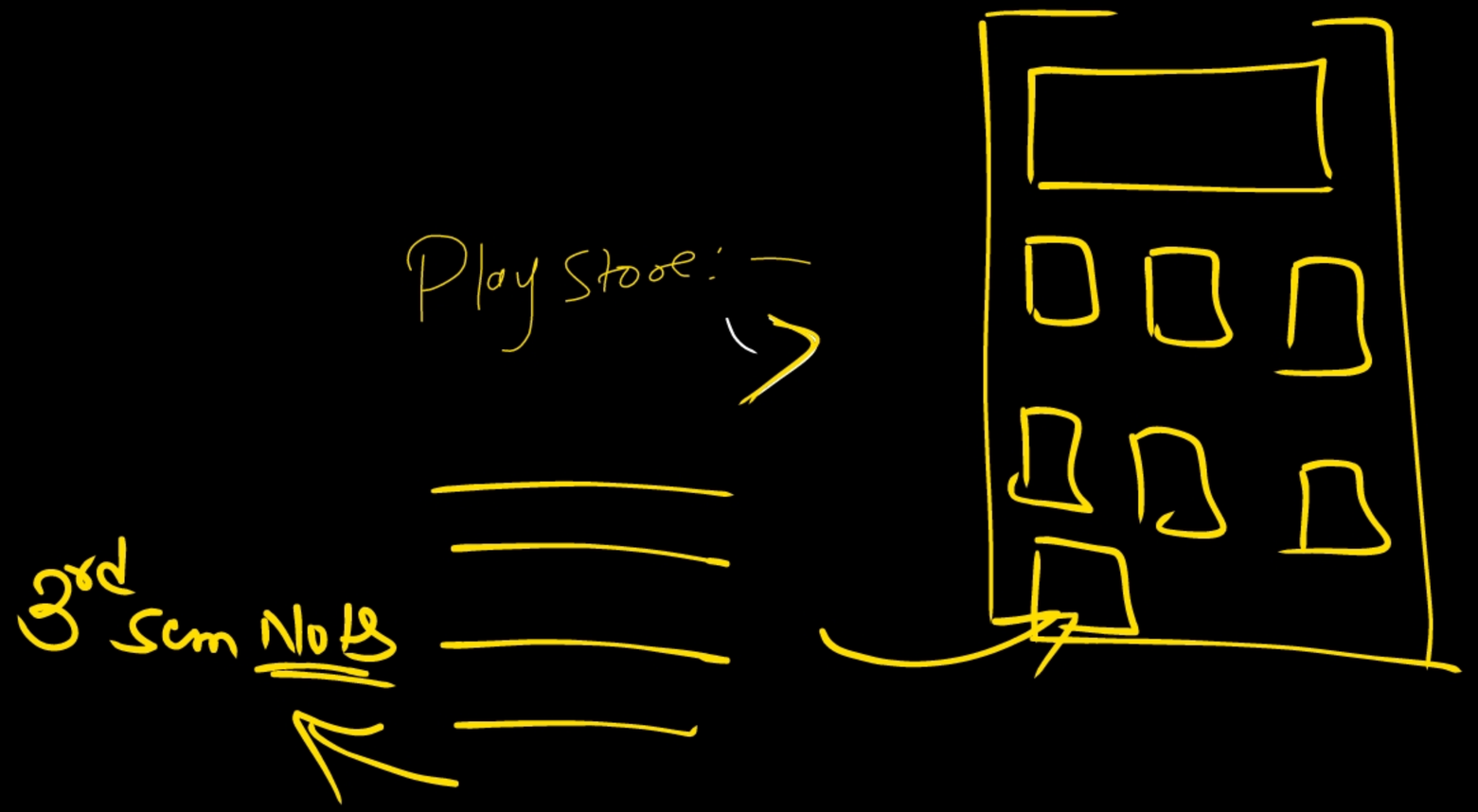
BEU

3nd Semester cse Branch

DATA STRUCTURE AND ALGORITHMS

ONE-SHOT


Play Store :-
3rd Scm Nots

LAKSHYA BATCH FOR CSE

Start Date: 18th May 2026

4TH SEMESTER

Er. Nadim | Er. Asif | Er. Hammad

Trusted by Thousands of CSE Aspirants

Offer Price:

₹1,799 ~~₹2,999~~ 40% OFF

Contact :- 9060601584

- Complete Syllabus
- Exam-Oriented Approach
- Regular Doubt Solving

ENROLL NOW!

Education PLUS+

Unit 1:0

INTRODUCTION

WHAT IS DATA STRUCTURE?

A data structure is a particular way of organising data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks.

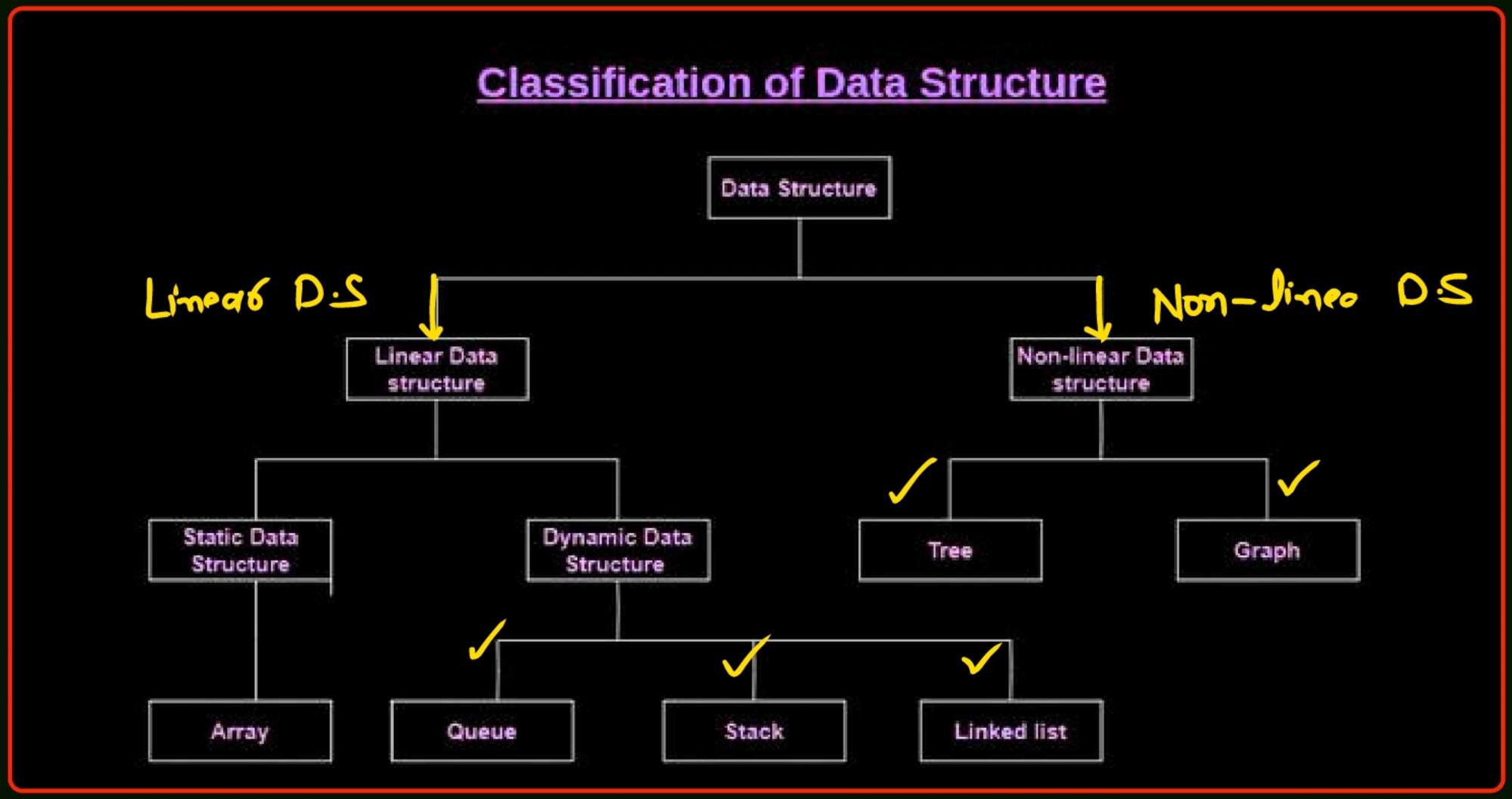
WHAT IS DATA STRUCTURE?

- The structure of the data and the synthesis of the algorithm are relative to each other. Data presentation must be easy to understand so the developer, as well as the user, can make an efficient implementation of the operation.
- Data structures provide an easy way of organising, retrieving, managing, and storing data.

LIST OF THE NEEDS FOR DATA.

- Data structure modification is easy.
- It requires less time.
- Save storage memory space.
- Data representation is easy.
- Easy access to the large database

Classification of Data Structure



WHAT IS ANALYSIS OF ALGORITHMS?

- Analysis of algorithms is the process of studying how efficient an algorithm is in terms of time (speed) and space (memory usage). It helps us compare different algorithms and decide which one is best suited for solving a problem.

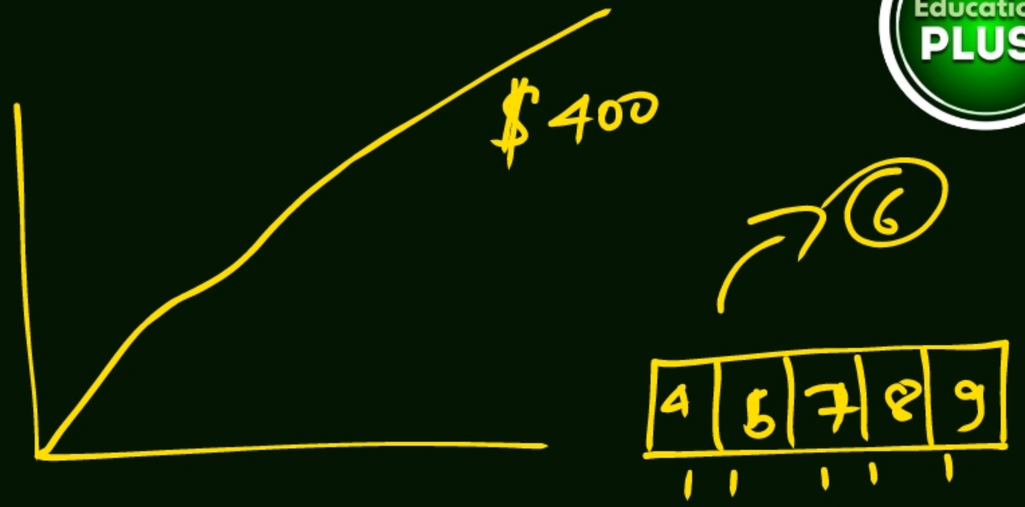
KEY POINT

- Purpose: To predict the performance of an algorithm before actually running it on a computer.
- Focus Areas:
- Time Complexity → How long the algorithm takes as input size grows.
- Space Complexity → How much memory it consumes.
- Method: Use mathematical tools like asymptotic notation (Big O, Big Ω, Big Θ) to describe growth rates

★ Master method

TYPES OF ANALYSIS

- Worst-case analysis
- Best-case analysis
- Average-case analysis

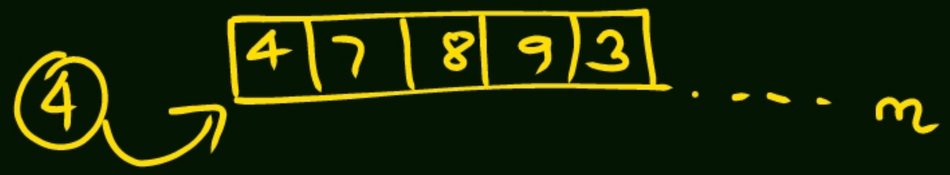


WORST-CASE ANALYSIS

- In the worst-case analysis, we calculate the upper bound on the running time of an algorithm. We must know the case that causes a maximum number of operations to be executed.
- For Linear Search, the worst case happens when the element to be searched (x) is not present in the array. When x is not present, the search() function compares it with all the elements of arr[] one by one.
- This is the most commonly used analysis of algorithms (We will be discussing below why). Most of the time we consider the case that causes maximum operations.

2.BEST CASE ANALYSIS

- In the best-case analysis, we calculate the lower bound on the running time of an algorithm. We must know the case that causes a minimum number of operations to be executed.
- For linear search, the best case occurs when x is present at the first location. The number of operations in the best case is constant (not dependent on n). So the order of growth of time taken in terms of input size is constant.



3. AVERAGE CASE ANALYSIS

- In average case analysis, we take all possible inputs and calculate the computing time for all of the inputs. Sum all the calculated values and divide the sum by the total number of inputs.
- Assuming all cases (including x not being in the array) are equally likely in linear search, the average case is computed by summing all cases and dividing by n+1n+1n+1, to account for the possibility that the element is absent.

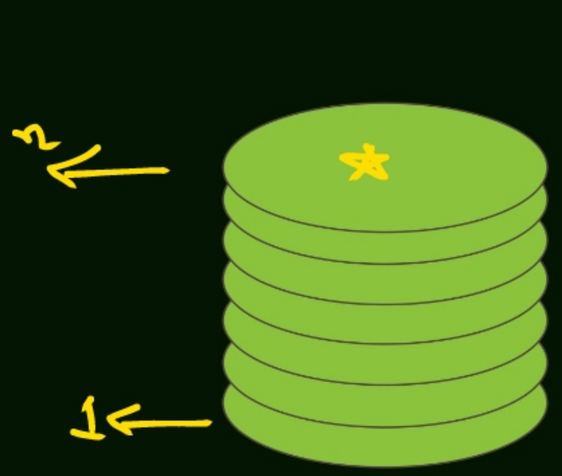
Education PLUS+

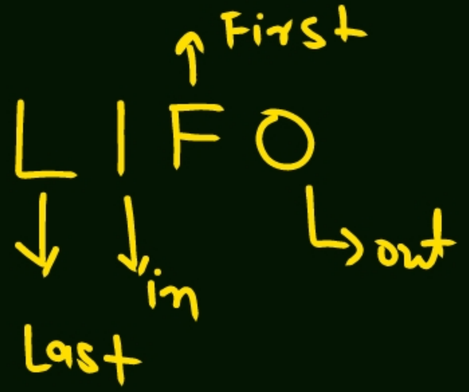
Unit 2:0

STACKS AND QUEUES

WHAT IS A STACK?

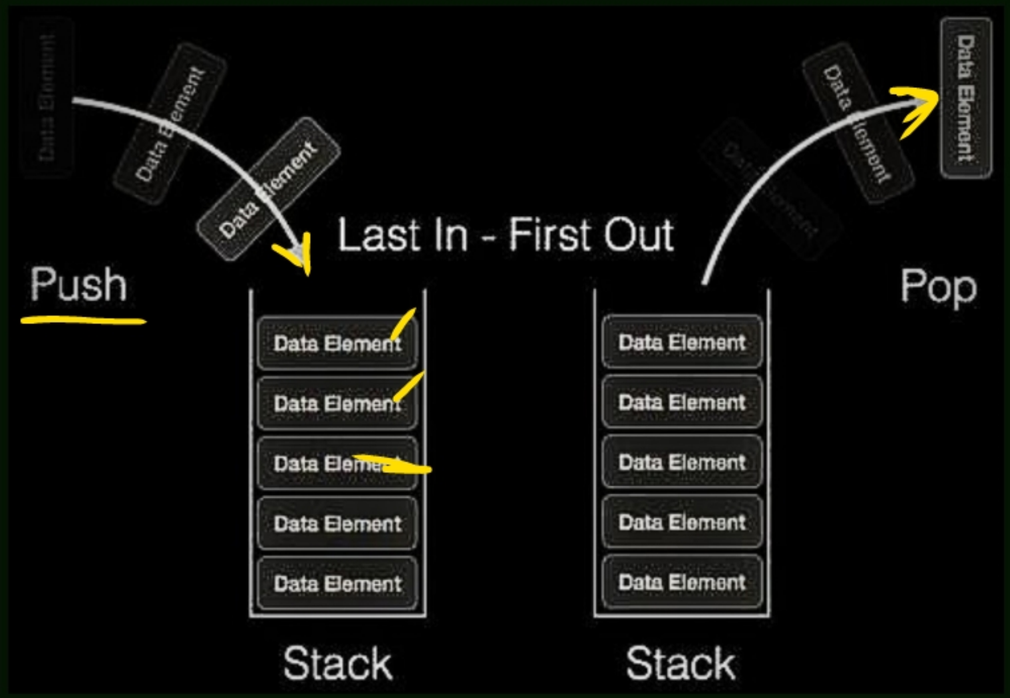
- A stack is a linear data structure where elements are stored in the LIFO (Last In First Out) principle where the last element inserted would be the first element to be deleted. A stack is an Abstract Data Type (ADT), that is popularly used in most programming languages. It is named stack because it has the similar operations as the real-world stacks, for example – a pack of cards or a pile of plates, etc.





STACK REPRESENTATION

- A stack allows all data operations at one end only. At any given time, we can only access the top element of a stack.



- The most fundamental operations in the stack ADT include: push(), pop(), peek(), isFull(), isEmpty().

STACK INSERTION: PUSH( )

- The push() is an operation that inserts elements into the stack. The following is an algorithm that describes the push() operation in a simpler way.

ALGORITHM

- 1. Checks if the stack is full.
- 2. If the stack is full, produces an error and exit.
- 3. If the stack is not full, increments top to point next empty space.
- 4. Adds data element to the stack location, where top is pointing.
- 5. Returns success.

STACK DELETION: POP( )

- The pop() is a data manipulation operation which removes elements from the stack. The following pseudo code describes the pop() operation in a simpler way.

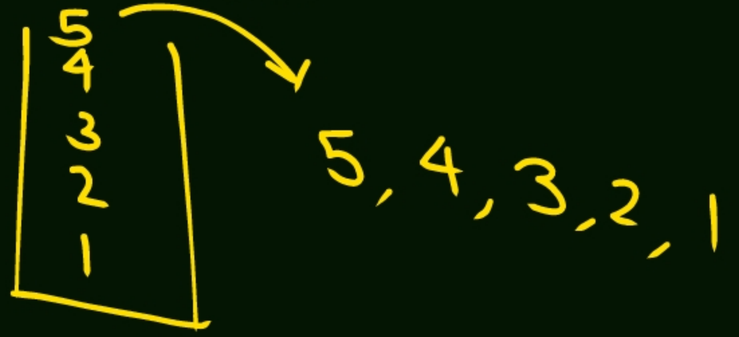
ALGORITHM

1. Checks if the stack is empty.
2. If the stack is empty, produces an error and exit.
3. If the stack is not empty, accesses the data element at which top is pointing.
4. Decreases the value of top by 1.
5. Returns success.

APPLICATION OF STACK

- Expression parsing:- stack help evalute and checks programming expression, ensuring balance parentheses.
- Backtracking: used in algorithum like maze-solving and "Eight Queens" puzzle
- Function calling: Manag function details during calls in programming Langeuage.
- undo feature: Implement undo in text editier and brouser
- syntex checking: compilers use stack to match syntax element like "if" with "else"

- Question: If the input sequesnces is 1,2,3,4,5 hen identify the worng stack permutation
  - 1. 3,5,4,2
  - 2,4,3,5,1
  - 4,3,5,2,1
  - 5,4,3,2,1



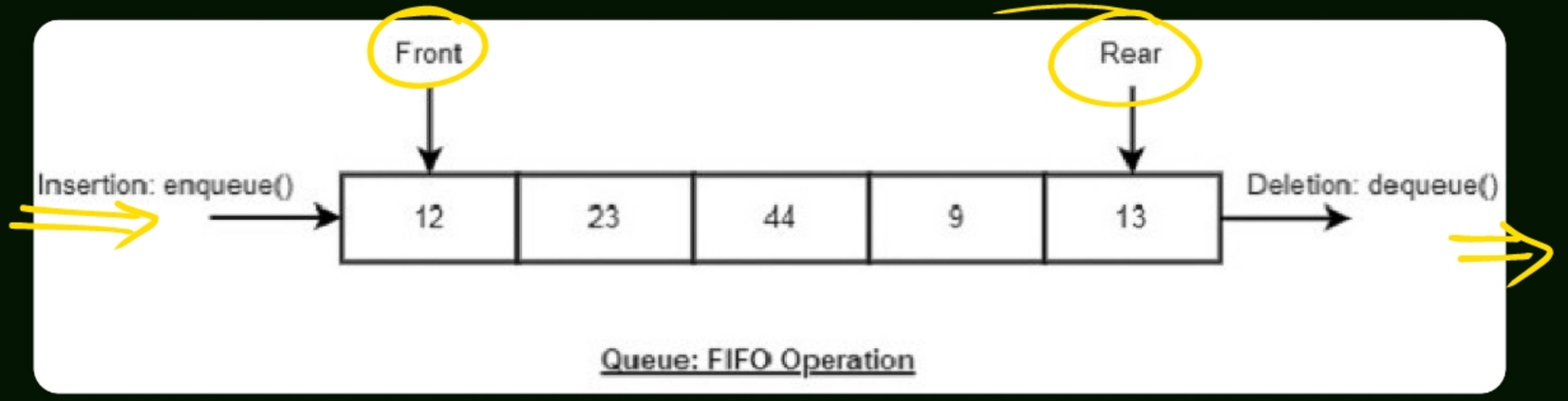
QUEUE DATA STRUCTURE

FIFO

- A queue is a linear data structure where elements are stored in the FIFO (First In First Out) principle where the first element inserted would be the first element to be accessed. A queue is an Abstract Data Type (ADT) similar to stack, the thing that makes queue different from stack is that a queue is open at both its ends. The data is inserted into the queue through one end and deleted from it using the other end. Queue is very frequently used in most programming languages.

REPRESENTATION OF QUEUES

- The most fundamental operations in the queue ADT include: enqueue(), dequeue(), peek(), isFull(), isEmpty(). These are all built-in operations to carry out data manipulation and to check the status of the queue.



Queue: FIFO Operation

FIFO

- The most fundamental operations in the stack ADT include: push(), pop(), peek(), isFull(), isEmpty().

QUEUE INSERTION OPERATION: ENQUEUE( )

- The enqueue() is a data manipulation operation that is used to insert elements into the stack. The following algorithm describes the enqueue() operation in a simpler way.

ALGORITHM

1. START 2. Check if the queue is full.
3. If the queue is full, produce overflow error and exit.
4. If the queue is not full, increment rear pointer to point the next empty space. 5. Add data element to the queue location, where the rear is pointing.
6. return success. 7. END

QUEUE DELETION OPERATION: DEQUEUE()

- The dequeue() is a data manipulation operation that is used to remove elements from the stack. The following algorithm describes the dequeue() operation in a simpler way

ALGORITHM

1. START
2. Check if the queue is empty.
3. If the queue is empty, produce underflow error and exit.
4. If the queue is not empty, access the data where front is pointing.
5. Increment front pointer to point to the next available data element.
6. Return success.
7. END

QUEUE – THE PEEK() OPERATION

- The peek() is an operation which is used to retrieve the frontmost element in the queue, without deleting it. This operation is used to check the status of the queue with the help of the pointer.

ALGORITHM

1. START
2. Return the element at the front of the queue
3. END

QUEUE – THE ISFULL() OPERATION

- The isFull() operation verifies whether the stack is full.

ALGORITHM

1. START
2. If the count of queue elements equals the queue size, return true
3. Otherwise, return false
4. END

QUEUE – THE ISEMPTY() OPERATION

- The isEmpty() operation verifies whether the stack is empty. This operation is used to check the status of the stack with the help of top pointer.

ALGORITHM

1. START
2. If the count of queue elements equals zero, return true
3. Otherwise, return false
4. END

TYPE OF QUEUE

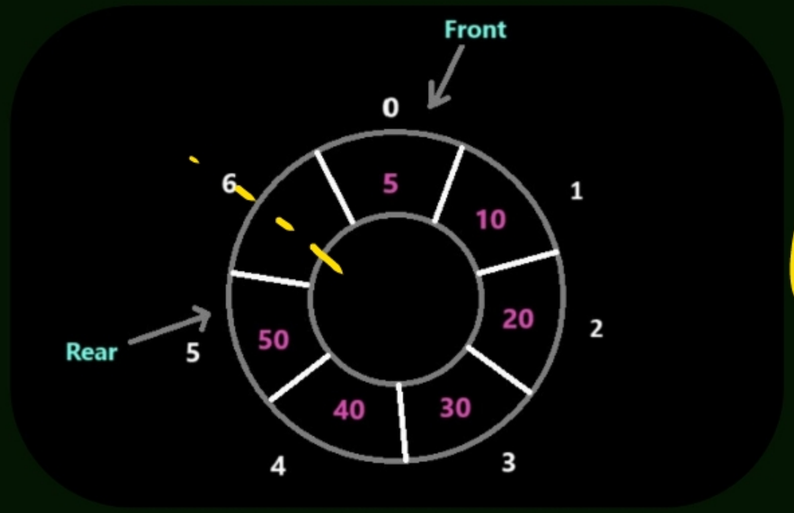
- Simple Queue, Circular queue, Priority queue

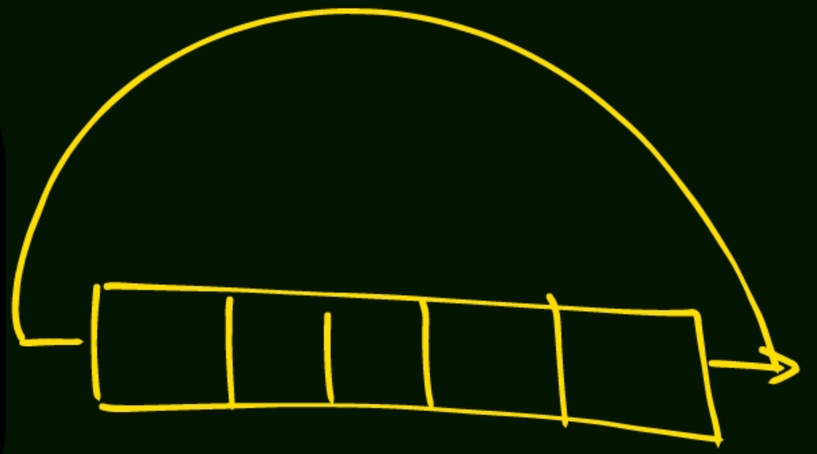
WHAT IS CIRCULAR QUEUE?

- A circular queue is a type of queue in which the last position is connected back to the first position to make a circle. It is also known as a Ring Buffer. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using a circular queue, we can use the space to insert elements.
- It is a linear data structure that follows the FIFO mechanism. The circular queue is a more efficient way to implement a queue in a fixed size array. In a circular queue, the last element points to the first element making a circular link.

REPRESENTATION OF CIRCULAR QUEUE

- This is the representation of a circular queue, where the front is the index of the first element, and the rear is the index of the last element.





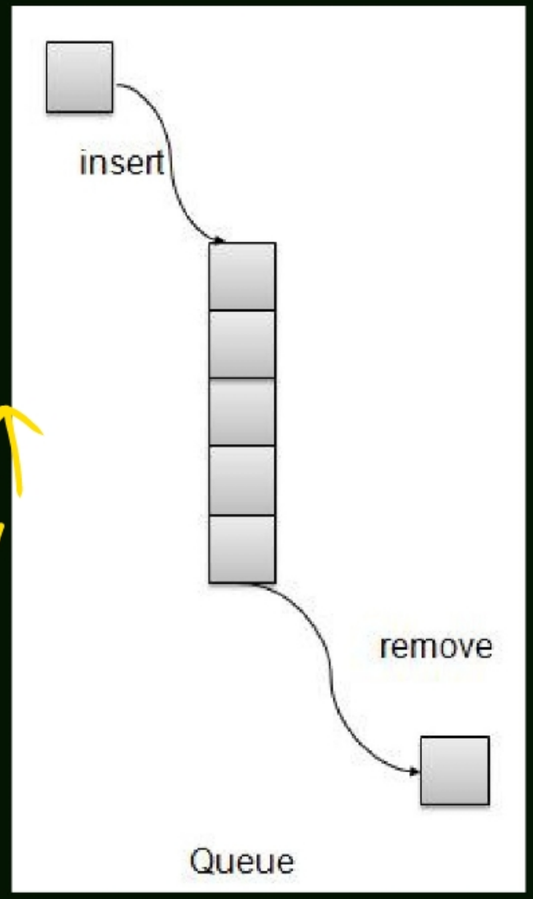
- Enqueue: Insert an element into the circular queue.
- Dequeue: Delete an element from the circular queue.
- Front: Get the front element from the circular queue.
- Rear: Get the last element from the circular queue.

PRIORITY QUEUE

- Priority Queue is more specialized data structure than Queue. Like ordinary queue, priority queue has same method but with a major difference. In Priority queue items are ordered by key value so that item with the lowest value of key is at front and item with the highest value of key is at rear or vice versa. So we're assigned priority to item based on its key value. Lower the value, higher the priority. Following are the principal methods of a Priority Queue.

BASIC OPERATIONS

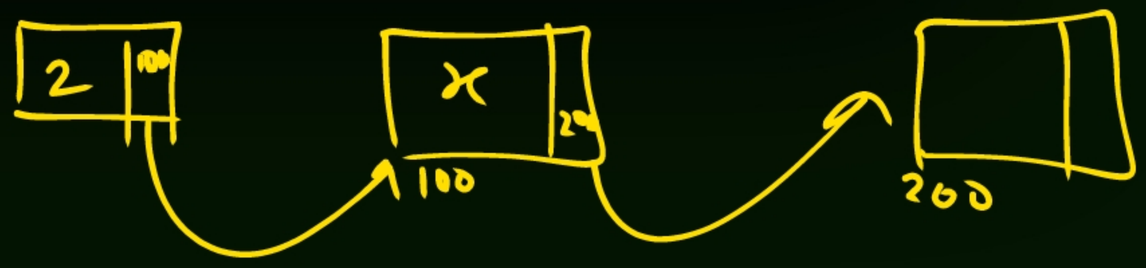
- insert / enqueue – add an item to the rear of the queue.
- remove / dequeue – remove an item from the front of the queue.



Education PLUS+

Unit 3:0

LINKED LIST

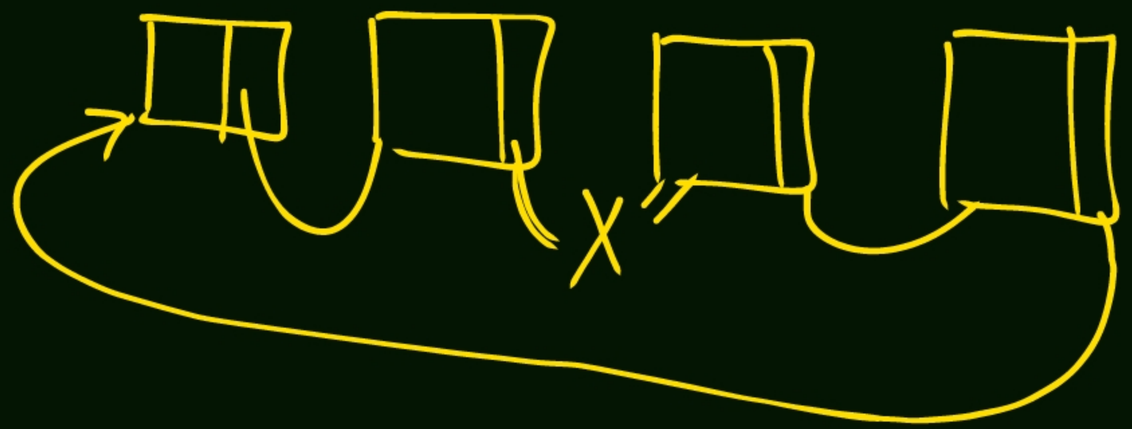


LINKED LIST

- A linked list is a dynamic data structure that consists of element called nodes which are connected in a linear sequence Each mode contains two parts: data and a reference to next node.
- The first part is the information part of the mode which can store any type of information such as integer characters or objects.
- The secound part called linked field or next pointer field, contains the address of the next node of to list
- The pointer of the last node contains a null pointer which is an invalid address (0 or negative value)
- The linked also contain a list pointer variable called start/first/head which contain the address of the first node in the list
- A special case is the list that has no node, such a list is called null list or empty list is denoted by a null pointer in the variable start/ fist /head.

ADVANTAGE OF LINK LIST:-

- Dynamic size and efficient memory usege:- Linked list can easily grow or shrink allowing for efficient memory allocation and reduced waste as element are added or removed.
- Fast insertion or deletion: operation like inserting or removing element can be performed in constant time (O(1)) if the position is known, offering batter performance compared to array -based structure.
- versalility: Linked list can be adapted to various type (singly, doubly, circular)



DISADVANTAGE OF LINKLIST

- Slower access time:- linked list have a higher time complexity for element access (O(n)) compared to array, as element must be accessed sequentially from the head of the list.
- Memory overhead: Each node in a linked list rquired additional memory to store the refrance (or pointer) to the next node increasing the overall memory usage compared to array-based structure.
- Pointer manipulation: Implementing linked list involves managing pointer, which can increase code complexity and lead to potential issue, such as memory leaks or segmentetion faults, if not handled carefully

HEADER LINK LIST

- A header linked list is a variation of a standard linked list that includes a special node called the header node at the begining of list
- The header node does not store any data; instead, it serves a fixed reference point that simplifies some operation on the linked list like insecrting or deleting element at the begining of list

HEADER LINK LIST

- The header node is always present, even when the list is empty
- The header node's next pointer points to the first actual data node in the list or Null if the list is empty.
- The header node can also store Metadata about the list, such as its length although this it not a requirment.

TYPE OF LINKED LIST

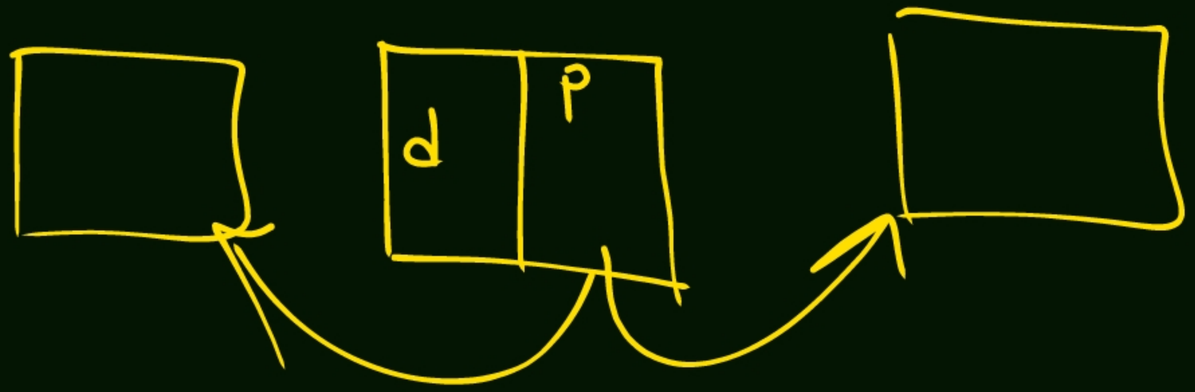
- circular linked list
- doubly linked list

CIRCULAR LINKED LIST

- A single circular linked list is a variation of singly linked list in which the last node points back to the first node creating a loop or circular structure.
- The primary different between a standard singly linked list and singly circular linked list is that the last node's "Null" points refers to the first node in the list, rather then being Null.

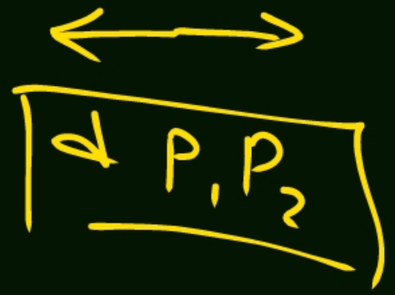
DOUBLY LINK LIST

- A doubly linked list is a a data sturucture in which each node contains a data element and two pointers, one pointing to the previous and the other pointer to the next node (The "next" pointer) in sequence.
- This bidirectional linking allows for easier traversal and manipulating of the list in both forward and backward direction as well as simplifying some operation such as insertion or deletion of nodes at any position in the list.



SOME FEATURES OF DOUBLY LINKED LIST:-

- Each node has two pointers "next" pointing to the subsequent node and previous pointing to the precding node in the list
- The first node's "previous" pointers and the last node's next pointer are set to null indicating the beginning and end of the list respectively.
- Doubly linked list allows for easier traversal and manipulation in both forward and backward direction compared to singly linked list.
- Doubly linked list consume more memory then singly linked list due to the additional previous pointer.



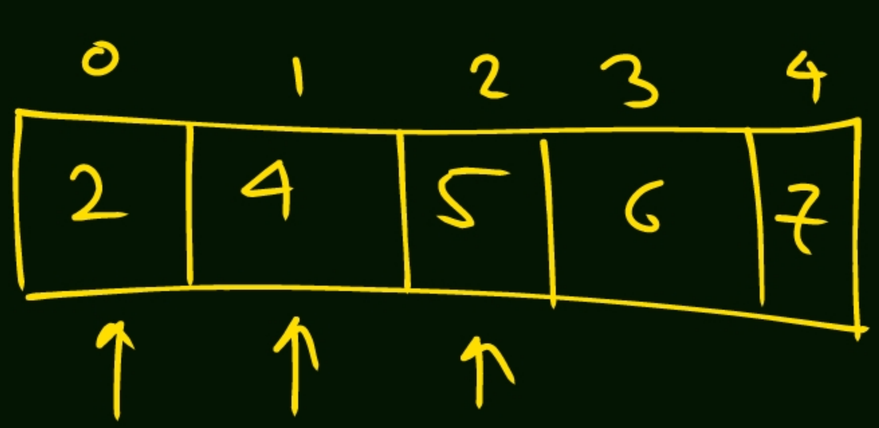
Unit 4:0

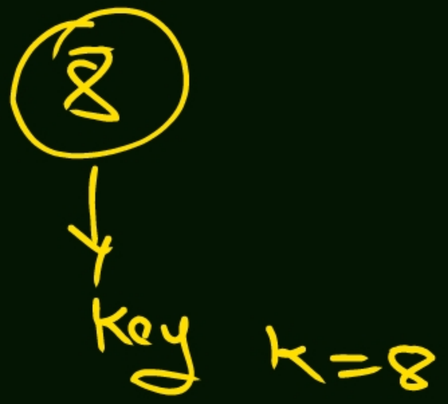
SEARCHING,SORTING & HASHING

- Linear Search
- Binary Search
- Insertion sort
- Bubble shot
- selection shot
- Merg shot
- Heap Shot

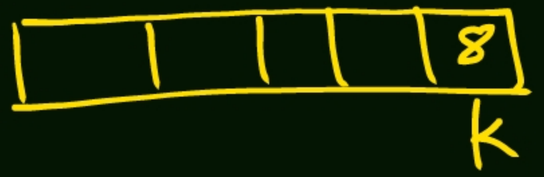
LINEAR SEARCH & BINARY SEARCH

- Linear search is a type of sequential searching algorithm. In this method, every element within the input array is traversed and compared with the key element to be found. If a match is found in the array the search is said to be successful; if there is no match found the search is said to be unsuccessful and gives the worst-case time complexity.





TIME COMPLEXITY



Worst Case:

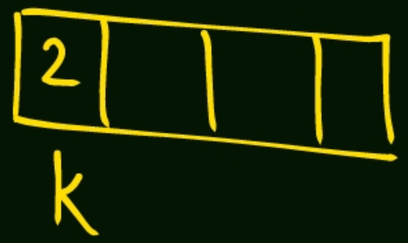
- Occurs when the target element is at the very end of the list or not present at all. The algorithm must perform comparisons to reach this conclusion.

Average Case:

- Occurs when the target is somewhere in the middle of the list. On average, the algorithm performs about comparisons, which still simplifies to linear time.

Best Case:

- Occurs when the target element is the very first item in the list, requiring only a single comparison

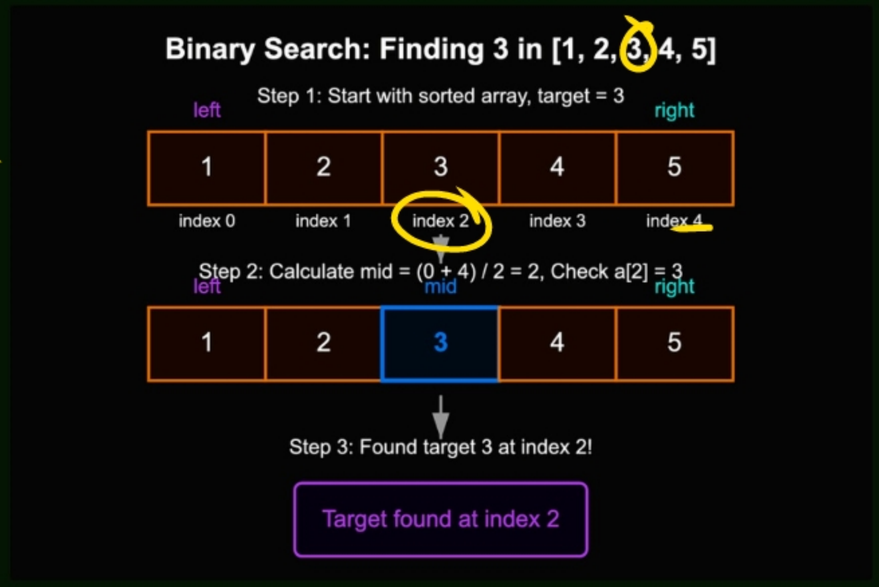


BINARY SEARCH

- Binary Search is an efficient algorithm used to find the position of a target element in a sorted list or array. Instead of checking each element one by one (like linear search), it repeatedly divides the search interval in half, drastically reducing the number of comparisons.
- Best case: O(1) → Target is the middle element on the first try.
- Average/Worst case: O(\log n) → Each step halves the search space

$\frac{4}{2} = 2$

key = mid



OBJECTIVE AND PROPERTIES OF SORTING ALGORITHMS

- The main objective of sorting algorithms is to arrange data in a specific order (ascending or descending) so that it becomes easier to:
  - Search efficiently (binary search requires sorted data).
  - Organize information for better readability and processing.
  - Optimize other algorithms (many algorithms run faster on sorted data).
  - Enable data analysis (like finding medians, duplicates, or ranges quickly).

Sorting algorithms can be compared based on several important properties:

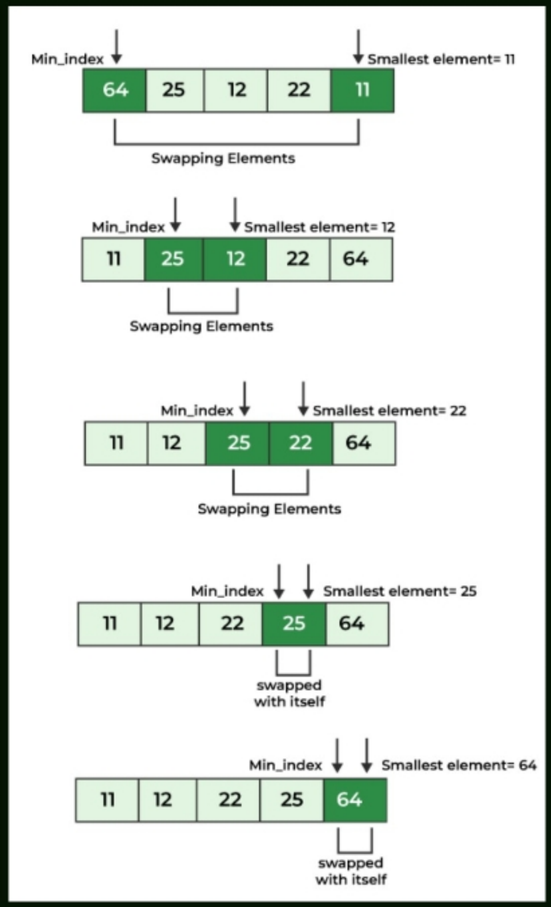
- Time Complexity
- How fast the algorithm sorts data.
- Example: Quick Sort → average O(n\log n), Bubble Sort → O(n^2).
- Space Complexity How much extra memory is required.
- Example: Merge Sort needs extra space, while Heap Sort sorts in place.

OBJECTIVE AND PROPERTIES OF SORTING ALGORITHMS

- Stability: A stable sort keeps equal elements in their original order.
- Example: Merge Sort is stable, Quick Sort is not (by default).
- Adaptability:Some algorithms perform better if the data is already partially sorted.
- Example: Insertion Sort adapts well to nearly sorted data.
- Comparison-based: Decide order by comparing elements (Quick Sort, Merge Sort).
- Non-comparison-based: Use other techniques like counting or hashing (Counting Sort, Radix Sort).

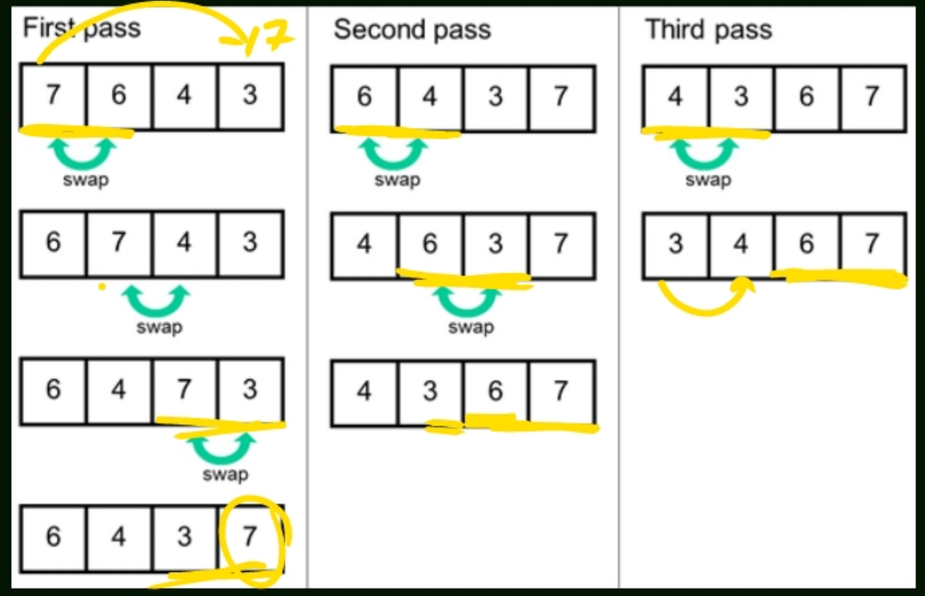
SELECTION SHOT

- Idea: Repeatedly find the smallest (or largest) element and place it in the correct position.
- Steps:
- Find the minimum element in the unsorted part.
- Swap it with the first unsorted element.
- Repeat until the list is sorted.
- Complexity: O(n^2) comparisons, not adaptive.
- Property: Simple, but inefficient for large datasets



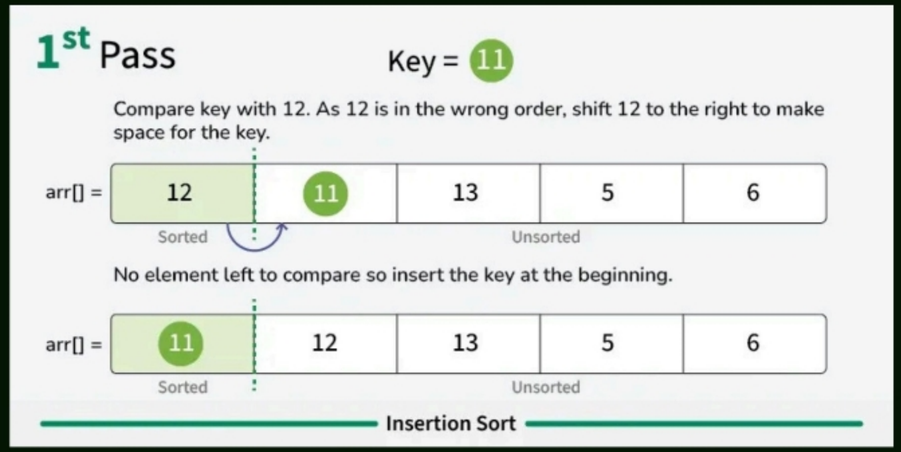
BUBBLE SHOT

- Idea: Repeatedly swap adjacent elements if they are in the wrong order.
- Steps:
- Compare each pair of neighbors.
- Swap if needed.
- Keep bubbling the largest element to the end.
- Complexity: O(n^2), but best case O(n) if already sorted.
- Property: Stable, easy to understand, but slow.



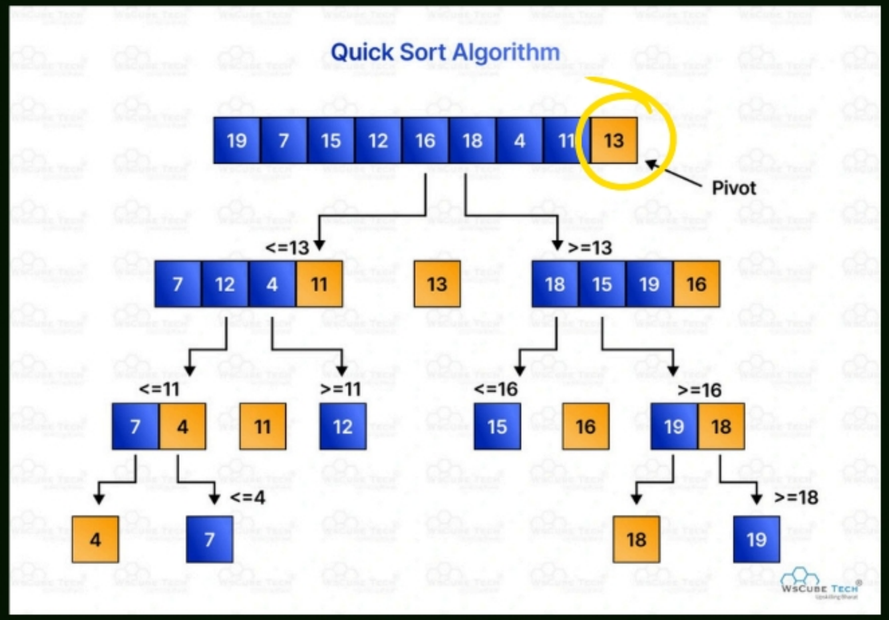
INSERTION SHOT

- Idea: Build the sorted list one element at a time by inserting each new element into its correct position.
- Steps:
- Take the next element.
- Compare with sorted part.
- Insert it in the right place.
- Complexity: O(n^2), but best case O(n) for nearly sorted data.
- Property: Stable, adaptive, good for small or nearly sorted datasets.



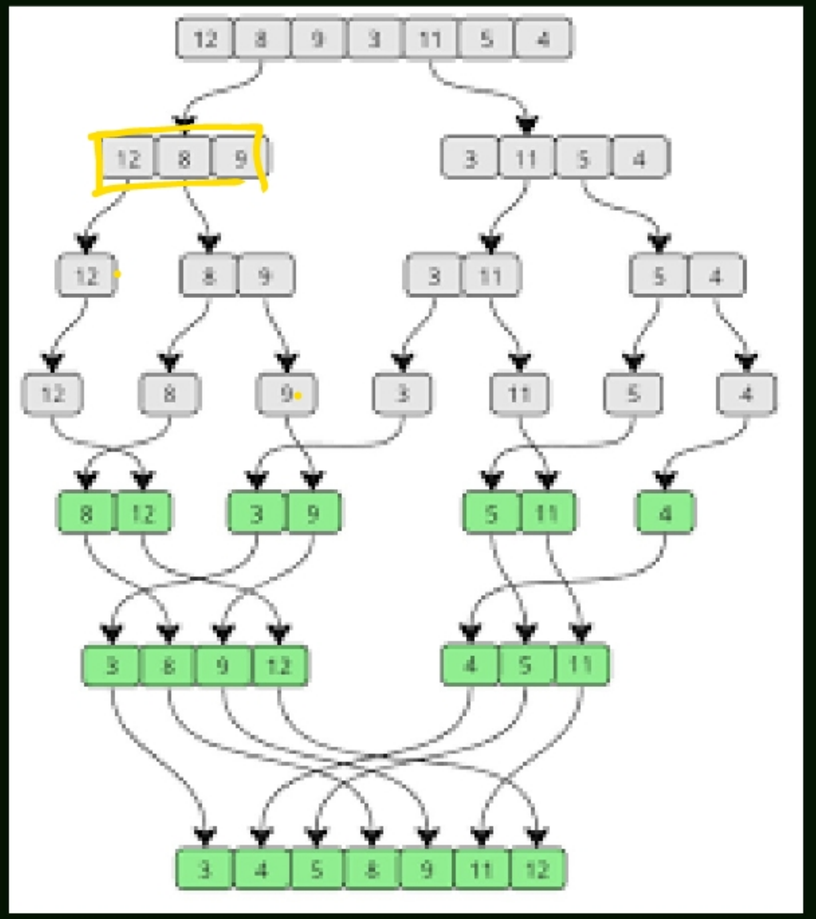
QUICK SHOT

- Idea: Divide and conquer using a pivot.
- Steps:
- Pick a pivot element.
- Partition array into two halves (less than pivot, greater than pivot).
- Recursively sort each half.
- Complexity: Average O(n\log n), worst case O(n^2).
- Property: Very fast in practice, widely used.



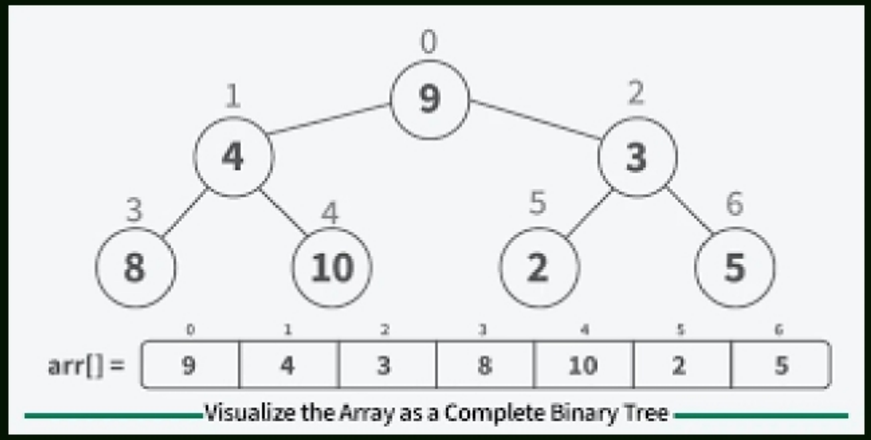
MERG SHOT

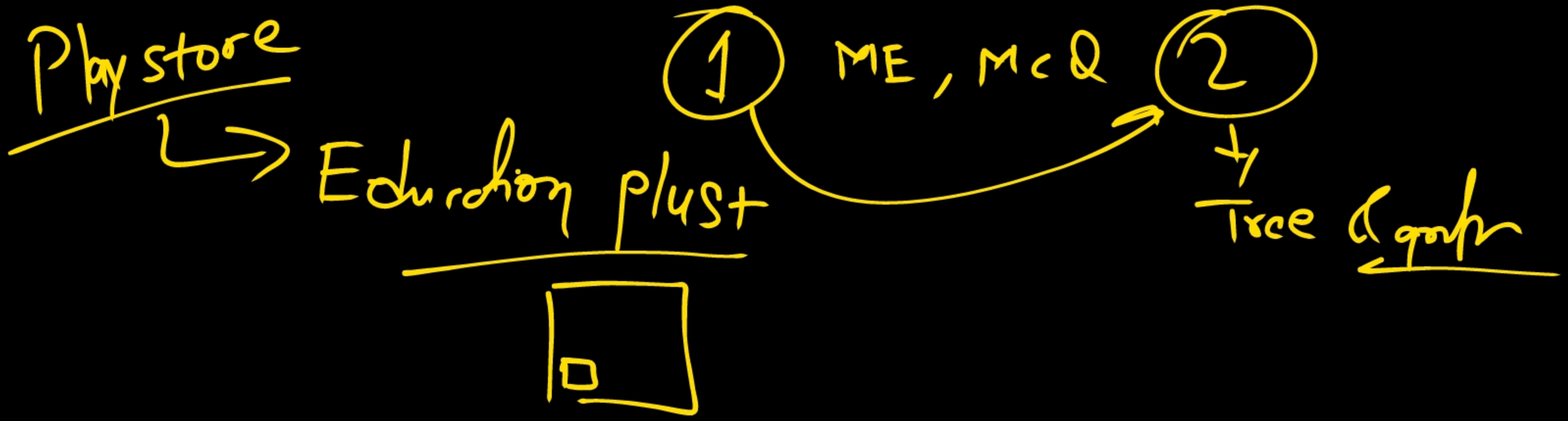
- Idea: Divide and conquer by splitting and merging.
- Steps:
- Split array into halves until single elements remain.
- Merge halves back together in sorted order.
- Complexity: Always O(n\log n).
- Property: Stable, predictable, but needs extra memory.



HEAP SHOT

- Idea: Use a binary heap structure to sort.
- Steps:
- Build a max-heap.
- Swap root (largest) with last element.
- Reduce heap size and heapify again.
- Complexity: O(n\log n).
- Property: In-place, not stable, efficient for large datasets




Playstore
Education plust
1
ME, McQ
2
Tree Graph

Thank You!